

**Use Case** diagram represents the system's functionality and shows users how software components connect each other logically and diagrammatically such as design.

Components:

1. **System.** It details a set of actions between actors and the data.  
 Notation: a rectangle with name on top of it.  
 Content: all use cases inside the rectangle.
2. **Use Case.** It represents a functional unit of a large application  
 Notation: horizontally shaped oval and is located inside the rectangle  
 Goals: interactions and actions between events, actors, and the data lead to the end

result

3. **Actor.** the entity that interacts with the subject  
 Notation is the "stick man"  
 Types: primary actor—using the use case  
 secondary actor – maintaining or providing service to the use case
4. **Relationship.** Associations between the actors and use cases – extend and include  
 A use case can be linked to multiple actors and vice versa.  
 Therefore-----> Multiplicity of use case / actor.
  - a. The multiplicity of use case--- a use case can be associated with multiple Actors i.e. View-Courses' is associated with two actors-'New-User' and 'Registered-User
  - b. The multiplicity of an Actor--- an association represented by a number
    - i. Multiplicity zero – It means the use case may have an instance of no actor.
    - ii. Multiplicity One – It means one actor is a must for the use case
    - iii. Multiplicity greater than 1 – means there can be multiple actors involved in a use case instance concurrently or at different points of time or sequentially. But rare

To-do list.

- 1) Project broken down into multiple small functionalities

Understand the complex large project and break it down into multiple functionalities and start documenting the detail of each functionality.

- 2) Identify the goal and prioritize it

Start listing each functionality identified with the goal to be achieved by the functionality.

Prioritize the identified functionality as per the business deliverable plan.

- 3) Functionality Scope

Understand the scope of the functionality and draw the system boundary.

Identify all the use cases that need to be part of the system to achieve the goal.

List all the actors (users and services) that have a role in the system. An actor can be a human, internal, and external application that can interact with the functionality.

#### #4) Identify relationship and association

Have clarity in the relationships and interdependency between use cases and actors.

#### #5) Identify Extension and Inclusion Use cases

List all the use cases with extension or Include a use case for it.

#### #6) Identify Multiplicity

Find multiplicity of Use cases and Actors, if any.

#### #7) Naming Use Case and actors

Follow a standard in naming the use cases and actors. The name should be self-explanatory.

The name referred to for a specific user/use case should be the same across the whole project.

A brief detail of use case functionality and the actors with access to the use case should be summarized under a specific section in the document.

#### #8) Important note points

Clarify and highlight important points using Notes without overburdening the use case with notes.

#### #9) Review

Review and validate the document before starting the drawing of the use cases.

Example—Class Registration System.

1. Analysis:



## 2. Draw Use Case Diagram: Step-by-Step Guideline